

TS-DM: A Time Segmentation-Based Data Stream Learning Method for Concept Drift Adaptation

Kun Wang^{1b}, Jie Lu^{1b}, *Fellow, IEEE*, Anjin Liu^{1b}, and Guangquan Zhang^{1b}

Abstract—Concept drift arises from the uncertainty of data distribution over time and is common in data stream. While numerous methods have been developed to assist machine learning models in adapting to such changeable data, the problem of improperly keeping or discarding data samples remains. This may result in the loss of valuable knowledge that could be utilized in subsequent time points, ultimately affecting the model's accuracy. To address this issue, a novel method called time segmentation-based data stream learning method (TS-DM) is developed to help segment and learn the streaming data for concept drift adaptation. First, a chunk-based segmentation strategy is given to segment normal and drift chunks. Building upon this, a chunk-based evolving segmentation (CES) strategy is proposed to mine and segment the data chunk when both old and new concepts coexist. Furthermore, a warning level data segmentation process (CES-W) and a high-low-drift tradeoff handling process are developed to enhance the generalization and robustness. To evaluate the performance and efficiency of our proposed method, we conduct experiments on both synthetic and real-world datasets. By comparing the results with several state-of-the-art data stream learning methods, the experimental findings demonstrate the efficiency of the proposed method.

Index Terms—Concept drift, data stream, ensemble learning, time segmentation.

I. INTRODUCTION

CONCEPT drift reflects the change of data distribution in uncertain environments [1], [2], [3]. This phenomenon is prevalent in the real world, such as weather changes, user interest drift, stock market fluctuation, and so on. When concept drift occurs under uncertainty, machine learning models without updating may fail to adapt to the changeable data over time, resulting in subpar performance. Concept drift can be divided into four categories according to its frequency and severity: 1) sudden drift; 2) incremental drift; 3) gradual drift; and 4) reoccurring drift [1], [4]. In some cases, different types and severities of concept drift may coexist or reoccur within the same data stream. In such complex situations, blindly

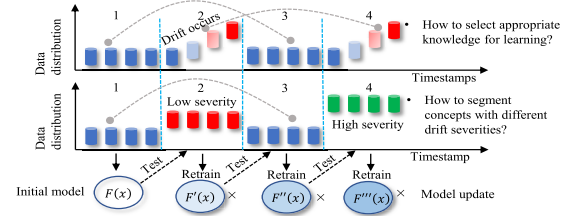


Fig. 1. Illustration of the research problem. When concept drift occurs with different situations, the traditional retrain method (initially train a model, then test and retrain it at each data chunk.) may not make its learned historical knowledge match the demand of learning new incoming concept, especially when data changes with different drift severity. How to segment and select appropriate streaming data and handle different drift severities?

learning data chunks without considering the selection and extraction of useful knowledge seems insufficient. As a consequence, the knowledge acquired at the current moment may not be promptly aligned with the requirements of the learning task in the next moment. Therefore, it becomes necessary to enhance the machine learning model's adaptive capabilities. Some previously research works given solutions like historical data usage [5] and data sample filtering [6]. However, it faces the challenge of sufficiently selecting appropriate data that aligns with the learning and prediction of the model at the next time point. Therefore, the appropriate usage and segmentation of data stream in relation to the model's upcoming learning tasks become necessary to mitigate performance decay.

Segmentation excels at time series data mining in previous research works [7], [8], but few of them address the issue of data stream segmentation, especially under the influence of concept drift. Different from time series data, which focuses on analyzing historical observations at fixed time intervals, data stream mining deals with continuous and evolving data arriving in real-time. And the uncertain and unpredictable change in streaming data distribution makes the segmentation process become more complex, as depicted in Fig. 1. On the one hand, the segmentation process in data stream learning should consider handling concept drift to support model learning at the next time point, further extension in traditional time series segmentation strategies, like window selection [7], [9] and clustering [10], is necessary. Instead, suitable segmenting and combining drift data and normal data may help reduce error and enhance generalization. On the other hand, when concept drift reoccurs in a periodicity, the historical data may be reused to support the learning model. The idea of abandoning all the previous data in the traditional retraining process [1] is

Manuscript received 29 December 2023; revised 2 April 2024 and 21 June 2024; accepted 10 July 2024. Date of publication 12 August 2024; date of current version 8 October 2024. This work was supported by the Australian Research Council under Grant DP200100700 and Grant FL190100149. This article was recommended by Associate Editor H. Wang. (Corresponding author: Jie Lu.)

The authors are with the Australia Artificial Intelligence Institute, Faculty of Engineering and Information Technology, University of Technology Sydney, Sydney, NSW 2007, Australia (e-mail: kun.wang@uts.edu.au; jie.lu@uts.edu.au; anjin.liu@uts.edu.au; guangquan.zhang@uts.edu.au).

This article has supplementary material provided by the authors and color versions of one or more figures available at <https://doi.org/10.1109/TCYB.2024.3429459>.

Digital Object Identifier 10.1109/TCYB.2024.3429459

suboptimal and may lead to model failure in subsequent time points. Motivated by this, this article considers about how to segment and select the appropriate streaming data to match the prediction demands of model learning flexibly when concept drift occurs.

Another issue is the different drift situations, which should also be segmented appropriately to reduce interference. Most of the related works only focus on change detection without considering various drift situations [9], [11]. Motivated by this, we consider two aspects: 1) drift coexistence and 2) drift type. First of all, segmentation should be more flexible when both normal and drift data coexist in the same data chunk. Second, maintaining model generalization requires appropriately segmenting and selecting data with different types of drift. As mentioned in [12], streaming data can be classified into three levels: 1) normal level; 2) warning level; and 3) drift level. Simply segmenting and grouping the data may not fully assist the model in adapting to such a complex scenario. Therefore, the second target is to appropriately and selectively segment the streaming data to help adapt to different drift situations.

Furthermore, to maintain learning robustness, it is essential to consider the tradeoff between high and low-drift severity [13] when segmenting and selecting streaming data. During the segmentation and selection of streaming data, retaining some recent data can be beneficial, but the decision to keep or discard concepts depends on the severity of the drift. For instance, retaining old concepts can enhance generalization when the drift has low severity in the subsequent time point, while some older concepts may need to be discarded if the new concepts exhibit high-drift severity. Blindly keeping or deleting old concepts can lead to a model that performs poorly in handling both high and low-severity concept drift. To further bolster learning robustness, it is also crucial to selectively segment streaming data while considering this issue.

To solve these problems, in this article, we propose a time-segmentation-based data stream learning method (TS-DM) to dynamically segment and learn streaming data under concept drift. The main contributions of this article are as follows.

- 1) A chunk-based segmentation strategy for concept drift learning has been proposed to segment the data chunks into normal and drift sets. The process of segmenting and grouping data chunks to separate training sets supports model retraining at each time point.
- 2) To deal with the situation where both normal and drift data coexist in a data chunk, a chunk-based evolving segmentation (CES) strategy for concept drift learning has been proposed for timely location of the segment point. This strategy selects the data with more flexibility and reduces the learning error significantly.
- 3) Based on the CES method, a warning level data segmentation process is proposed and embedded to help the model adapt to different types of concept drift, called CES-W. This enables the timely establishment of the segment point of warning level data.
- 4) A high-low-drift tradeoff handling process has been proposed; the optimized version is called the TS-DM. The proposed method has ideal learning performance on both real-world and synthetic datasets.

The remainder of this article is organized as follows. Section II summarizes the recent works. The research problem statement and the proposed approaches are introduced in Sections III and IV. And Section V describes the experiment setting and discusses the results. Finally, a summary of the conclusion and our future study is given in Section VI.

II. RELATED WORK

This section summarizes the existing concept drift learning methods and some previous research works on time segmentation learning.

A. Concept Drift Adaptation for Data Stream Learning

As a challenge in data stream learning, concept drift, which describes the uncertain changes in data distribution over time, attracts attention recently [1], [14]. For concept drift detection, methods like one-class drift detector (OCDD) [15] and self-training data streams (STDSs) [16] are recently proposed. In addition, there are also methods focus on detecting different types of concept drift [17]. For concept drift adaptation, many learning methods are aligned with ensemble methods due to its excellent performance [18], [19], [20], [21]. Compared with the single learning model, the ensemble model integrates the learning results of multiple base models, improving the final results' accuracy and robustness. Bagging and Boosting are two commonly used ensemble methods for concept drift learning. The previously mentioned adaptive random forests (ARFs) [22] is a classical bagging-based concept drift learning method. Furthermore, Oza Bagging [23] is also a popular ensemble learning method for data stream learning. Besides, Online Boosting [24] and the online RUSBoost (RUSB) [24] are boosting-based methods. Currently, methods like clustering [25], SVM [26], and fuzzy theory [27] have also been used in related areas.

In addition, different perspectives have been considered in concept drift learning. For example, aggregated Mondrian forest classifier (AMF) [28] is developed for online learning while incremental weighted ensemble (IWE) [29] is addressed for data stream learning. Furthermore, concept drift learning [30], [31] in multiple data streams have also been addressed.

B. Time Segmentation-Based Learning Methods

The segmentation method is early proposed and applied in the time series data to support the prediction task [9]. It aims to identify changing patterns in time series data for model learning and has been widely used in time series and online learning tasks. In previous studies, sliding window (SW) [9] is a classic method that sets segment points on time series according to whether the learning error exceeds the max error threshold. Many methods carry out time segmentation based on learning performance. First, according to the pattern of time series data, research [32] regards the time segmentation problem as an optimization problem and applies an evolutionary approach to deal with it. To deal with structured time series data, kernelized temporal cut (KTC) is a novel method of changepoint [11]. Furthermore, the clustering method has

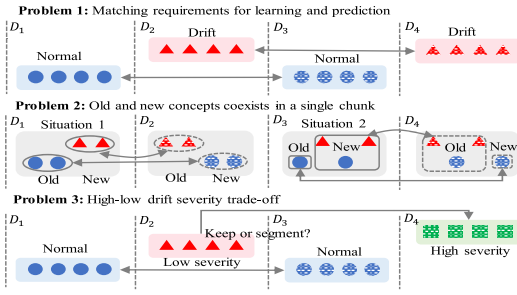


Fig. 2. Illustration of the problems of segmentation in data stream under concept drift. Problem 1 shows the requirement of matching between learning and prediction. Problem 2 focuses on the situation of old and new concepts coexisting in a single chunk. Problem 3 concerns the tradeoff of high-low drift severity.

also been used in segmenting time series data [10]. Recently, segmentation has been addressed in different learning tasks, like anomaly detection tasks [33], few-shot learning tasks [34], and multivariate time series [35].

Although these methods work well, there are still research gaps that are not considered. First, the streaming data may need to be preserved and segmented to match the demands of model learning at the next prediction task. Although time segmentation methods are proposed in some previous works, few of them consider and discuss the situation of data stream learning under concept drift. Furthermore, with different drift types and drift severities, the learning process should adapt over in time; it is needed to enhance learning generalization and robustness.

III. PROBLEM STATEMENT

This section gives a detailed description and statement of three identified problems, as shown in Fig. 2.

Consider three consecutive data chunks D_t , D_{t+1} , D_{t+2} at time points t , $t+1$, $t+2$, $D_t = \{x_{t,i}, y_{t,i}\}_{i=1}^n$, $D_{t+1} = \{x_{t+1,i}, y_{t+1,i}\}_{i=1}^n$, $D_{t+2} = \{x_{t+2,i}, y_{t+2,i}\}_{i=1}^n$, where n is the number of the data instance. Assume data distribution in D_t is normal but occurs concept drift in D_{t+1} . To deal with this, a learning model $F(x_t)$ is initially trained on chunk D_t and tested on chunk D_{t+1} , thus outputting the loss at time t and $t+1$, denote as $L(y_t, F(x_t))$ and $L(y_{t+1}, F(x_t))$. Under the influence of concept drift, the model performance may be degraded due to a lack of efficient adaptation, leading to an increased loss of the learning model, expressed as

$$L(y_{t+1}, F(x_t)) \geq L(y_t, F(x_t)) \quad (1)$$

which reflects the insufficient adaptation on D_{t+1} . Then, consider a scenario where the data distribution of D_{t+2} undergoes continual changes, experiences concept drift, or reverts to normal. In such case, even retraining the model on D_{t+1} may not perform well on D_{t+2} due to the uncertain changes. Hence, the first problem is how to select the appropriate data to align with the requirements of model prediction timely, and the definition is shown below.

Problem 1 (Matching Requirements for Learning and Prediction): Given consecutive data chunks $\{D_t, D_{t+1}, D_{t+2}\}$, D_t is normal while D_{t+1} occurs concept drift. The performance

of model $F(x)$ on D_{t+2} depends on its adaptability after learning on D_t and D_{t+1} sequentially. The loss of the model is

$$\mathcal{D} = \arg \min_{\mathcal{D}} L(y, F(x_{\mathcal{D}})) \quad (2)$$

where the $\mathcal{D}$ has

$$\begin{cases} \mathcal{D} = D_t, & \text{if } D_{t+2} \text{ is normal} \\ \mathcal{D} = D_{t+1}, & \text{if } D_{t+2} \text{ is drift.} \end{cases} \quad (3)$$

Example: Consider consecutive streaming data chunks in Fig. 2(a), D_1 is normal while D_2 occurs drift, thus the learning model trained on D_1 may not adapt to D_2 well. Likewise, retraining the model on D_2 may not ensure performance on D_3 and D_4 , which change again. Therefore, the matching requirement for learning and prediction is needed.

However, the coexistence of old and new concepts in a single data chunk may degrade the model performance. Therefore, the second problem is how to segment old concepts and new concepts in a single data chunk.

Problem 2 (Old-New Concepts Segmentation in A Single Chunk): Given data chunks $\{D_t, D_{t+1}, D_{t+2}\}$; chunk D_t contains old concepts while chunk D_{t+1} contains both old and new concepts. Appropriately segmenting old and new concepts with obvious differences is needed to help reduce the loss of model, denoted as

$$\eta = \arg \min_{\eta} L(y, F(\mathcal{D}_{\eta})), \eta \in [0, n] \quad (4)$$

where the η is the segmented point of old concepts and new concepts. The data selection of $\mathcal{D}_{\eta}$ depends on the drift situation of chunk D_{t+2} . If D_{t+2} is normal, $\mathcal{D}_{\eta} = \{x_{t+1,i}\}_{i=0}^{\eta}$, which contains the old concepts. Otherwise, the data containing new concepts can be chosen for learning.

Example: Consider consecutive streaming data chunks of situations 1 and 2 in Fig. 2(b), old and new concepts may coexist frequently in a single data stream. The model learned on D_1 might be impacted when tested on D_2 with hybrid concepts. So, appropriately segmenting and utilizing old and new concepts may help maintain model learning.

In addition, the data after segmenting should be selectively used for model learning, since concept drift may occur with different severity at the next time point. To reduce the influence of the historical data, the third problem is how to segment the data $\mathcal{D}$ again to keep the data at recent time points to support model adaptation.

Problem 3 (Robustness Requirement): Consider segmenting and combining the data with new concepts from two consecutive data chunks, $D_{t+1, \eta_{t+1}}$ and $D_{t+2, \eta_{t+2}}$, for model learning and prediction. If a new incoming chunk occurs concept drift with higher-drift severity, it is needed to consider keeping or deleting earlier collected data based on drift severity

$$\min\{L(F(D_{t+2, \eta_{t+2}})), L(F(D_{t+1, \eta_{t+1}} \cup D_{t+2, \eta_{t+2}}))\}. \quad (5)$$

Example: Consider consecutive streaming data chunks in Fig. 2(c), D_2 and D_4 contain drift with different severity, although combining them as a new train set increases variety, discard D_2 may be better when there is a significant difference between them, which will affect the model adaptation.

The following part of this article will present the learning methods corresponding to these three problems.

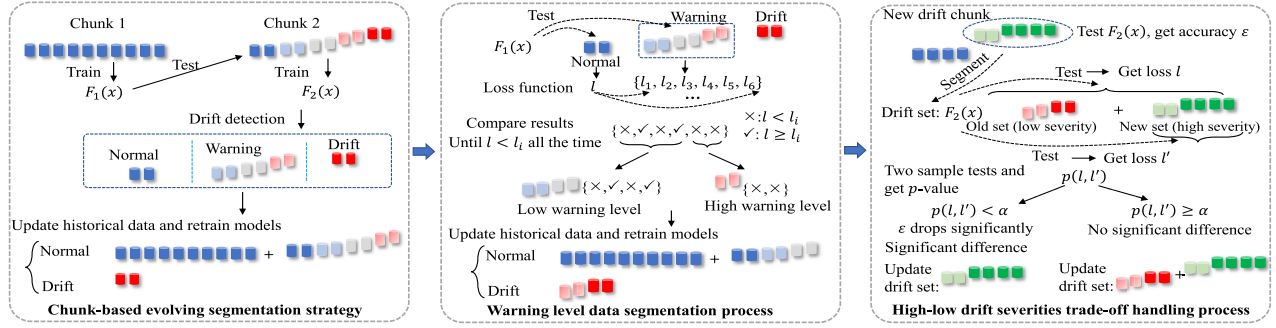


Fig. 3. Illustration of the detailed procedure of the proposed TS-DM. The CES strategy aims to divide the normal and drift data in chunk. The warning level data segmentation process tries to find and group data with high-warning levels. For the data with different drift severity, selectively segment it timely by handling the high-low-drift severity tradeoff.

IV. PROPOSED METHOD

This section develops a TS-DM to deal with data stream with concept drift. The basic framework of the proposed TS-DM is shown in Fig. 3.

A. Chunk-Based Segmentation Strategy

To address Problem 1, we propose a chunk-based segmentation strategy, which simply segments and groups data chunks into normal and drift sets, then choose the appropriate one for model learning.

1) *Preparing the Learning Models for Normal and Drifted Training Sets:* Consider a data stream that consists of a series of data chunks $\{D_t\}_{t=1}^T$, where t is the time step, n is the chunk size, denoted as $D_t = \{x_i, y_i\}_{i=1}^n$. Assuming there are three consecutive data chunks D_t, D_{t+1}, D_{t+2} , at time $t, t+1, t+2$, the data distribution in chunk D_t is normal but occurs concept drift in D_{t+1} , and the distribution of chunk D_{t+2} is unknown. To reduce the influence of concept drift, we initially segment chunks D_t, D_{t+1} as two separate training sets: 1) normal set and 2) drift set. We first train a model $F_1(x)$ on chunk D_t which focuses on normal data learning. Then, training another model $F_2(x)$ on chunk D_{t+1} to deal with the data occurs concept drift. (In this article, both $F_1(x)$ and $F_2(x)$ are used as Gradient Boosting model, which is an ensemble model that combines several base learners.) At time $t+2$, we separately test $F_1(x)$ and $F_2(x)$ on chunk D_{t+2} . If chunk D_{t+2} is normal, $F_1(x)$ may be better for prediction, and vice versa. Therefore, we choose the appropriate model according to their accuracy $\varepsilon_{F_1(x)}, \varepsilon_{F_2(x)}$. We then choose the results with the higher accuracy according to the principle of following the leader, with the average accuracy denoted as:

$$\varepsilon_{\text{ave}} = \frac{1}{T} \sum_{t=1}^T \max\{\varepsilon_{F_1(x),t}, \varepsilon_{F_2(x),t}\}. \quad (6)$$

2) *Updating the Training Sets:* To enhance adaptability, we update the training sets of these two models separately and sequentially. According to these two models' accuracy, if $\varepsilon_{F_1(x)} \geq \varepsilon_{F_2(x)}$, it indicates that model $F_1(x)$ works relatively well on chunk D_{t+2} ; thus, we combine chunks D_t and D_{t+2} as the new training set, so that model $F_1(x)$ can be retrained.

Algorithm 1: Chunk-Based Segmentation Strategy

Input : Data chunks D , learning model $F(x)$.
Output: Model prediction results.

```

1 When  $t = 1$ , train a model  $F_1(x)$  on  $D_1$ ;
2 for  $t = 2$  to  $T$  do
3   Test model  $F_1(x)$  on  $D_t$  and get accuracy  $\varepsilon_1$ ;
4   Train model  $F_2(x)$  on  $D_t$  and get accuracy  $\varepsilon_2$ ;
5   Collect the higher accuracy by  $\max(\varepsilon_1, \varepsilon_2)$ ;
6   if  $\varepsilon_1 \geq \varepsilon_2$  then
7     Update  $D_{\text{normal}}$  by Eq. (7) and keep the latest  $\mu$  samples;
8     if  $\varepsilon_{1,t} < \varepsilon_{1,t-1}$  then
9       Update  $D_{\text{normal}} \leftarrow D_t$ ;
10    end
11    Retrain  $F_1(x)$  on  $D_{\text{normal}}$ ;
12  end
13  if  $\varepsilon_1 < \varepsilon_2$  then
14    Update  $D_{\text{drift}}$  by Eq. (7) and keep the latest  $\mu$  samples;
15    if  $\varepsilon_{2,t} < \varepsilon_{2,t-1}$  then
16      Update  $D_{\text{drift}} \leftarrow D_t$ ;
17    end
18    Retrain  $F_2(x)$  on  $D_{\text{drift}}$ ;
19  end
20 end

```

If $\varepsilon_{F_1(x)} < \varepsilon_{F_2(x)}$, we combine chunks D_{t+1} and D_{t+2} as the new training set for model $F_2(x)$, expressed as

$$\begin{cases} \varepsilon_{F_1(x)} \geq \varepsilon_{F_2(x)}, & D_{\text{normal}} \leftarrow D_t \cup D_{t+2} \\ \varepsilon_{F_1(x)} < \varepsilon_{F_2(x)}, & D_{\text{drift}} \leftarrow D_{t+1} \cup D_{t+2}. \end{cases} \quad (7)$$

To maintain learning efficiency, we preserve the recent μ ($\mu = 1000$) numbers of data instances and discard the outdated ones by default, but only the latest data chunk would be kept when there is a decline in learning accuracy. The process of this strategy is shown in Algorithm 1.

B. Chunk-Based Evolving Segmentation Strategy

As described in Problem 2, the target is to handle the situation when old and new concepts coexist in a single chunk. Therefore, the CES strategy is proposed to address this issue.

1) *Segmenting Normal and Drift Data in Single Chunk:* When chunk D_{t+2} contains both normal and drift data, it is necessary to detect the drift data first. Here, we use a classical concept drift detection method (DDM) to detect drift data in chunk D_{t+2} . This method has two thresholds to describe data change: 1) warning level and 2) drift level. Warning level indicates no significant change in data, while

Algorithm 2: Identifying Drift Data in CES Strategy

Input : Data chunks D , chunk size n , learning model $F(x)$.
Output: Model prediction results.

```

1 When  $t = 1$ , train a model  $F_1(x)$  on  $D_1$ ;
2 for  $t = 2$  to  $T$  do
3   for  $i = 1$  to  $n$  do
4     Detect concept drift by DDM method;
5     Collect the data with index  $i_w$  where warning occurs;
6     Collect the data with index  $i_d$  where drift occurs.
7   end
8 end

```

drift level means the data changes significantly, triggering drift occurrence. Therefore, we detect it from the first data instance of chunk D_{t+2} ; if drift occurs at i_d th data instance, we group the preceding data into normal set and the subsequent data into drift set. If there is a high-drift frequency in a data chunk, this process only records the data instance where drift occurs the first time. That is, to say, once the first data instance in a chunk occurs drift and is detected, all the following data will be grouped into the drift set.

2) *Updating the Training Sets*: Then, we update the training sets of models $F_1(x)$, $F_2(x)$ by

$$\begin{cases} D_{\text{normal}} \leftarrow D_t \cup \{x_i, y_i\}_{i=1}^{i_d-1} \\ D_{\text{drift}} \leftarrow D_{t+1} \cup \{x_i, y_i\}_{i=i_d}^n \end{cases} \quad (8)$$

where $i_d \in [1, n]$, $\{x_i, y_i\}_{i=1}^{i_d-1} \in D_{t+2}$, $\{x_i, y_i\}_{i=i_d}^n \in D_{t+2}$. Apparently, the chunk-based segmentation strategy applies when $i_d = 1$ or $i_d = n$. Thus, the normal data and drift data in a single data chunk can be segmented appropriately. This strategy flexibly updates the training sets for models $F_1(x)$, $F_2(x)$ in real-time to further guarantee the learning performance, as shown in Algorithm 2.

C. Warning Level Data Segmentation Process

Simply grouping warning level data may affect model performance since it contains both low and high-warning level data, which needs further segmentation. As mentioned, in the data chunk D_{t+2} , the i_w and i_d are the points of warning level and drift level, so we get three subsets of data: 1) normal data; 2) warning data; and 3) drift data, denoted as $\{x_i, y_i\}_{i=1}^{i_w-1}$, $\{x_i, y_i\}_{i=i_w}^{i_d-1}$, $\{x_i, y_i\}_{i=i_d}^n$.

1) *High-Low-Warning Level Data Identification*: To identify the low-warning level data and high-warning level data in the warning data subset $\{x_i, y_i\}_{i=i_w}^{i_d-1}$, we test the model on the normal subset and calculate the loss function l_{normal} ; we also test the model on each of the data samples in the warning subset and calculate the loss function, denoted as $\{l_{i_w}, l_{i_w+1}, \dots, l_{i_d-1}\}$, where l_{i_w} is the loss when the model is tested on $\{x_i, y_i\}_{i=1}^{i_w}$, l_{i_w+1} is the loss when the model is tested on $\{x_i, y_i\}_{i=i_w+1}^{i_w+1}$, and the rest can be done in the same manner. Then, we greedily traverse this sequence of loss by l_{i_d-1} until the loss always satisfies

$$l_{i_w+\eta} > l_{\text{normal}}, \eta \in [i_w, i_d - 1]. \quad (9)$$

2) *Updating the Training Sets*: To segment the warning level data, we group the data sample after the $i_w + \eta$ th data instance into the drift subset. At the same time, we group the

Algorithm 3: TS-DM

Input : Data chunks D , chunk size n , learning model $F(x)$.
Output: Model prediction results.

```

1 When  $t = 1$ , train a model  $F_1(x)$  on  $D_1$ ;
2 for  $t = 2$  to  $T$  do
3   Drift detection and get  $i_w$  and  $i_d$ 
4   Test model  $F_1(x)$  on  $D_t$  and get accuracy  $\varepsilon_1$ ;
5   Train/test model  $F_2(x)$  on  $D_t$  and get accuracy  $\varepsilon_2$ ;
6   Collect the higher accuracy by  $\max(\varepsilon_1, \varepsilon_2)$ ;
7   Get the loss of  $F_1(x)$  on normal data  $l_{\text{normal}}$ ;
8   for  $i = i_w$  to  $i_d - 1$  do
9     Greedily traverse the sequence of loss function  $l_i$ ;
10    if there is  $l_{i_w+\eta} > l_{\text{normal}}$  then
11      Collect the index of  $i_w + \eta$ ;
12      Break;
13    end
14  end
15  if  $\varepsilon_1 \geq \varepsilon_2$  then
16    Update retraining set by Eq. (10)
17     $D_{\text{normal}} \leftarrow D_{\text{normal}} \cup \{x_i, y_i\}_{i=1}^{i_w-1} \cup \{x_i, y_i\}_{i=i_w}^{i_w+\eta}$ ;
18    if  $\varepsilon_{1,t} \leq l_{\text{lower}}$  then
19      Keep chunk  $D_{\text{normal}} \leftarrow D_t$ ;
20    end
21    Retrain  $F_1(x)$  on  $D_{\text{normal}}$ ;
22  end
23  if  $\varepsilon_1 < \varepsilon_2$  then
24    Update retraining set by Eq. (10)
25     $D_{\text{drift}} \leftarrow D_{\text{drift}} \cup \{x_i, y_i\}_{i=i_w+\eta+1}^{i_d-1} \cup \{x_i, y_i\}_{i=i_d}^n$ ;
26    Test the model  $F_2(x)$  on updated  $D_{\text{drift}}$  and get loss  $l_{\text{old}}$ ;
27    Test the model  $F_2(x)$  on  $\{x_i, y_i\}_{i=i_w+\eta+1}^{i_d-1} \cup \{x_i, y_i\}_{i=i_d}^n$  and
28    get loss  $l_{\text{new}}$ ;
29    Two-sample tests by Eq. (11)  $p_{\text{value}} \leftarrow p(l_{\text{old}}, l_{\text{new}})$ ;
30    Get accuracy  $\varepsilon_{t-1}$  and  $\varepsilon_t$ ;
31    if  $p_{\text{value}} < \alpha$  and  $\varepsilon_t < \varepsilon_{t-1} - \beta \varepsilon_{t-1}$  then
32       $D_{\text{drift}} \leftarrow \{x_i, y_i\}_{i=i_w+\eta+1}^{i_d-1} \cup \{x_i, y_i\}_{i=i_d}^n$ ;
33    end
34    Keep the latest  $\mu$  samples and retrain  $F_2(x)$  on  $D_{\text{drift}}$ .
35  end
36 end

```

data sample before the $i_w + \eta$ th data instance in the warning subset in the normal subset since it does not always increase the loss function. The training sets can be updated as

$$\begin{cases} D_{\text{normal}} \leftarrow D_t \cup \{x_i, y_i\}_{i=1}^{i_w-1} \cup \{x_i, y_i\}_{i=i_w}^{i_w+\eta} \\ D_{\text{drift}} \leftarrow D_{t+1} \cup \{x_i, y_i\}_{i=i_w+\eta+1}^{i_d-1} \cup \{x_i, y_i\}_{i=i_d}^n \end{cases} \quad (10)$$

where $\eta \in [i_w, i_d - 1]$. By greedily calculating and traversing the loss on the warning subset, the high-warning level data and low-warning level data can be segmented according to whether the loss function always increases or decreases. This strategy is called CES-W, and the process is outlined in Algorithm 3.

D. High-Low-Drift Severities Trade-Off Handling Process

As we mentioned in Problem 3, the updated training sets D_{normal} and D_{drift} for models $F_1(x)$, $F_2(x)$ play different roles. For updating training set D_{normal} , we calculate a 95% confidence interval ($l_{\text{lower}}, l_{\text{upper}}$) for the learning accuracy in the past time points, and keeping the latest data chunk once accuracy significantly declines. For updating D_{drift} , we separately collect two sets of loss iteratively by testing the learning model on the old and new concepts with different drift severity, then using two-sample tests to calculate the p_{value} . Once the learning accuracy declines and the p_{value} is lower than the significant level ρ ($\rho = 0.01$), keeping the new

concepts and discarding the old ones. Otherwise, a fixed size is set to preserve the latest μ instances as default. The detailed procedures are listed below:

1) *High-Low-Drift Severity Data Recognition*: According to (10), the D_{drift} consists of D_{t+1} and $\{x_i, y_i\}_{i=i_w+\eta}^{i_d}$ of D_{t+2} . Therefore, we test model $F_2(x)$ on D_{t+1} and the updated D_{drift} , then output the loss function of each iteration. Thus, we can get two sets of loss function $l_{\text{old}} = \{l_{\text{old},m}\}_{m=1}^M$, $l_{\text{new}} = \{l_{\text{new},m}\}_{m=1}^M$, M is the total number of iterations (base learners) of the Gradient Boosting model. To find the significant difference, two-sample K-S tests is applied to calculate the p_{value} , denoted as

$$p_{\text{value}} \leftarrow p(l_{\text{old}}, l_{\text{new}}). \quad (11)$$

2) *Updating Drift Set via the Significant Level of Drift Severity*: We compare the p_{value} with the significance level ρ , which is a commonly used statistical indicator to verify a significant difference. If $p_{\text{value}} < \rho$, it reflects that there is significant difference between D_{t+1} and the updated D_{drift} . At the same time, it is also necessary to observe whether the model accuracy ε_{t+2} at $t+2$ has been affected significantly. If the changed data causes a relatively lower decrease in accuracy, D_{t+1} can be used for model training to help maintain generalization. Otherwise, the D_{t+1} should be given up. Therefore, we set the model accuracy at $t+1$ as ε_{t+1} , and we manually set a parameter β to measure the maximum severity of accuracy be affected, denoted as $\beta\varepsilon_{t+1}$, $\beta \in (0, 1)$. If the accuracy at $t+2$ has

$$\varepsilon_{t+2} < \varepsilon_{t+1} - \beta\varepsilon_{t+1} \quad (12)$$

the training set D_{t+1} should be given up, so the updated D_{drift} can be expressed as

$$D_{\text{drift}} \leftarrow \{x_i, y_i\}_{i=i_w+\eta+1}^{i_d-1} \cup \{x_i, y_i\}_{i=i_d}^n. \quad (13)$$

In contrast, there is no significant difference between D_{t+1} and the updated D_{drift} , and they can be combined together as the updated D_{drift} , as shown in Algorithm 3 and Fig. 3. Thus, this process will help the model generalize well on sudden drift with lower severity and improve the adaptation ability on sudden drift with higher severity.

E. Theoretical Analysis

Given two consecutive data chunks D_t , D_{t+1} , D_t contains both new and old concepts, denoted as $P' = \alpha P + (1 - \alpha)Q$, $\alpha \in (0, 1)$, while D_{t+1} follows distribution Q . P represents the old concepts and Q represents the new concepts [36]. We initially train a model on chunk D_t without time segmentation process, and then test it on chunk D_{t+1} .

The proposition in [36] gives a bound representation of the learning model under concept drift. That is, for any $\delta > 0$, with probability at least $1 - \delta$, the generalization error R of the learning model $F(x)$ can be written as

$$R_Q(y_Q, F(x)) \leq \hat{R}_{P'}(y_{P'}, F(x)) + \sqrt{\frac{\log \frac{1}{\delta}}{2n}} + \alpha d(Q, P') + \Delta \quad (14)$$

where n is the number of data samples, $\hat{R}$ is the empirical error, the term $\alpha d(Q, P')$ represents the drift severity, and $\Delta = \int [l(y_Q, F(x)) - l(y_{P'}, F(x))] \phi_{P'}(x) dx$. The smaller the drift severity, the value of this term approaches 0, denoted as

$$\lim_{d(Q, P') \rightarrow 0} \alpha d(Q, P') + \Delta \approx 0. \quad (15)$$

Based on this proposition, we segment the chunk D_t and update the training set, then test the model on chunk D_{t+1} , the model loss can be written as

$$R'_Q(y_Q, F(x)) \leq \hat{R}_{(1-\alpha)Q}(y_{(1-\alpha)Q}, F(x)) + \sqrt{\frac{\log \frac{1}{\delta}}{2(1-\alpha)n}} + \underbrace{\alpha d(Q, Q) + \Delta}_{\approx 0}. \quad (16)$$

The number of training data has been reduced to $(1 - \alpha)n$, and the current influence of concept drift $\alpha d(Q, Q) + \Delta$ gradually approaches 0 as the drift severity decreases. Now, we can compare the model loss before and after the time segmentation process by

$$\begin{aligned} R_Q - R'_Q &\leq \left\{ \frac{\hat{R}_{\alpha P} + \hat{R}_{(1-\alpha)Q}}{\hat{R}_{P'}} + \sqrt{\frac{\log \frac{1}{\delta}}{2n}} + \alpha d(Q, P') + \Delta \right\} \\ &\quad - \left\{ \hat{R}_{(1-\alpha)Q} + \sqrt{\frac{\log \frac{1}{\delta}}{2(1-\alpha)n}} \right\} \\ &= \underbrace{\hat{R}_{\alpha P} + \left(1 - \sqrt{\frac{1}{1-\alpha}}\right) \left(\sqrt{\frac{\log \frac{1}{\delta}}{2n}}\right)}_{\text{item1} \leq 0} \\ &\quad + \underbrace{\alpha d(Q, P') + \Delta}_{\text{item2} \geq 0}. \end{aligned} \quad (17)$$

The results show that the time segmentation process can help reduce the model error when drift severity satisfies

$$|\text{item2}| \geq |\text{item1}|. \quad (18)$$

It illustrates that the proposed segmentation method can help maintain the model performance when drift severity increases obviously. Based on this, appropriately keeping some data with low-drift severity can help obtain learning robustness.

F. Time Complexity Analysis

For the training process, a Gradient Boosting model is trained at each time point. We assume the time complexity of a base learner of the Gradient Boosting model is $\mathcal{O}(h)$. There are M base learners that are initially trained. Thus, the time complexity is $\mathcal{O}(Mh)$. Since we separately train two models to deal with normal data and drift data, respectively, but in chunk-based segmentation strategy, we only retrain one of them at each time point, the time complexity is $\sigma \mathcal{O}(Mh) + (T - \sigma) \mathcal{O}(Mh)$, where T is the total time steps. Also, the data size at each time point is different, giving a variable v denoting the

data size, $v \in [n, \mu]$, so the time complexity of chunk-based segmentation strategy is

$$\sigma \mathcal{O}(vMh) + (T - \sigma) \mathcal{O}(vMh). \quad (19)$$

For the CES strategy, we also need to train two models to handle normal data and drift data, so the time complexity is $2T\mathcal{O}(vMh)$. In addition, the time complexity of drift identification is $\mathcal{O}(nMh)$, where n is the chunk size. Therefore, the time complexity of the CES strategy is

$$2T\mathcal{O}(vMh) + \mathcal{O}(nMh). \quad (20)$$

In the process of time segmentation for warning level data, the size of the warning subset at each time point is $i_d - i_w$, so the time complexity of this process is $\mathcal{O}(T(i_d - i_w))$. Finally, the model is tested on both earlier drift data and newly added drift data at each time point. So, the time complexity is $\mathcal{O}(2T)$. Therefore, the time complexity of our proposed TS-DM is

$$2T\mathcal{O}(vMh) + \mathcal{O}(nMh) + \mathcal{O}(T(i_d - i_w)) + \mathcal{O}(2T). \quad (21)$$

V. EXPERIMENTS

This section describes the experiment procedures, datasets, benchmark methods, and results.

A. Experiment Procedures

In the experiment, we use the prequential test-then-train mode for evaluating the proposed TS-DM on streaming datasets. The procedures are shown below.

Step 1: To verify the proposed chunk-based segmentation strategy, we tested it on synthetic and real-world datasets.

Step 2: To evaluate and discuss the CES-W strategy, we tested it on synthetic and real-world datasets.

Step 3: To deal with different types of concept drift, we evaluated the TS-DM by comparing with chunk-based segmentation, CES, CES-W, and other benchmark methods.

Step 4: To further verify the effectiveness of the proposed TS-DM, we compared it with the traditional Sliding Window-based segmentation method.

Step 5: To show the results of runtime analysis, we summarized and compared the time costs results.

B. Benchmark Method Description

We first compare the TS-DM with our proposed methods. The Gradient Boosting model from the Sklearn package is the learning model with the default parameter setting. We separately test the following proposed learning strategies.

- 1) *Chunk-Based Segmentation Strategy* deals with concept drift by chunk-based time-segmentation process.
- 2) *CES Strategy* segments data instances of a single data chunk to deal with concept drift.
- 3) *CES-W Strategy* adds a warning level data segmentation process to the proposed CES method.
- 4) *TS-DM Based on Sliding Window (SW)* handles concept drift by using CES-W and Sliding Window.
- 5) *TS-DM* enhances model robustness by using CES-W strategy and also embedding the high-low-drift severities tradeoff handling process.

TABLE I
DESCRIPTIONS OF BENCHMARK DATASETS

Synthetic data	#Sample	#Feature	#Class	#Chunk size	#Drift type
S1 (visual analysis)	12,000	2	2	-	Incremental
S2 (visual analysis)	12,000	2	2	-	Sudden
SEAA	10,000	3	2	100	Sudden
RTG	10,000	10	2	100	No
RBF	10,000	10	2	100	Incremental
RBFr	10,000	10	2	100	Incremental
AGRa	10,000	10	2	100	Sudden
HYP	10,000	10	2	100	Incremental
LEDA	10,000	24	10	100	Sudden
LEDg	10,000	24	10	100	Gradual
Sine	100,000	4	2	100	Sudden
Waveform	100,000	21	3	100	No
Real-world data	#Sample	#Feature	#Class	#Chunk size	#Drift type
Electricity	45,312	8	2	100	Unknown
Weather	18,159	8	2	365	Unknown
Usenet1	1,500	99	2	40	Unknown
Usenet2	1,500	99	2	40	Unknown
Spam	9,324	500	2	100	Unknown
EEG	14,980	14	2	100	Unknown
Airline	539,383	7	2	100	Unknown
Powersupply	29,928	2	24	100	Unknown
PokerHand	1,000,000	10	10	100	Unknown
Coverttype	581,012	54	7	100	Unknown
INSECTSi	452,044	33	6	100	Incremental
INSECTSa	355,275	33	6	100	Sudden

In addition, we also choose eight state-of-the-art benchmark methods for comparison.

- 1) *Streaming Random Patches (SRP)* [37] is an ensemble-based data stream learning method. We choose the random subspace mode for learning in the experiment.
- 2) *Robust Soft Learning Vector Quantization (RSLVQ)* [38] combines RSLVQ and momentum-based stochastic gradient.
- 3) *Adaptive Random Forest (ARF)* [22] is an ensemble method for data stream learning, which is designed based on the adaptive window and random forest.
- 4) *Online Under-OVER-Bagging (Online UOB)* [24] focuses on class imbalance in the data stream.
- 5) *OnlineBoost (OB)* [24] is the online version of the boosting framework, which aims to handle data stream.
- 6) *RUSBoost (RUSB)* [24] embeds the sampling techniques as a postprocessing step before each iteration.
- 7) *Aggregated Mondrian Forest Classifier (AMF)* [28] is an online random forest based on Mondrian Forests.
- 8) *Incremental Weighted Ensemble (IWE)* [29] is a chunk-based ensemble method for data stream learning.

All methods have the same chunk size on the same datasets. Motivated by research works [39], [40], [41], we select metrics of accuracy, macro F1 score, MCC score, macro precision score, macro recall score, and time costs for evaluation.

C. Data Description

1) *Synthetic Datasets:* First, we simply generate two synthetic datasets under Gaussian distribution with sudden and incremental drift to visualize the time-segmentation process. Each dataset has 12 000 data instances and two classes. The data information is shown in Table I.

Second, we select eight synthetic datasets that are generated by MOA [42] to simulate different types of concept drift, as

TABLE II
AVERAGE ACCURACY AND MACRO F1 SCORE WITH THE SYNTHETIC DATASETS (%), TS-DM GOT A HIGHER-AVERAGE PERFORMANCE

Accuracy	SEAA	RTG	RBF	RBFr	AGRa	HYP	LEDA	LEDg	Sine	Waveform	AvgAcc	AvgRank
CS	84.42 ± 0.4	69.89 ± 6.0	85.20 ± 1.7	81.32 ± 1.7	82.84 ± 0.9	78.29 ± 0.4	66.85 ± 0.6	65.91 ± 0.4	94.51 ± 0.0	83.71 ± 0.1	79.29(3)	4.3
CES	84.34 ± 0.5	69.80 ± 5.9	84.98 ± 1.6	81.26 ± 1.8	82.98 ± 0.8	78.14 ± 0.5	66.53 ± 0.5	65.83 ± 0.4	94.51 ± 0.1	83.73 ± 0.1	79.21(4)	5.0
CES-W	84.41 ± 0.4	69.90 ± 6.0	85.10 ± 1.6	81.33 ± 1.7	83.54 ± 0.9	78.09 ± 0.7	66.88 ± 0.3	65.95 ± 0.4	94.48 ± 0.0	83.69 ± 0.1	79.33(2)	4.4
TS-DM	85.33 ± 0.4	72.18 ± 5.9	86.51 ± 1.6	82.26 ± 1.6	85.77 ± 1.6	79.77 ± 0.6	67.89 ± 0.6	66.98 ± 0.6	95.97 ± 0.1	84.69 ± 0.1	80.73(1)	2.3
SRP	84.89 ± 0.3	66.66 ± 4.1	76.01 ± 2.2	74.41 ± 1.7	75.06 ± 1.5	72.01 ± 0.7	54.38 ± 3.7	53.94 ± 3.4	98.83 ± 0.1	33.29 ± 0.1	68.94(10)	7.1
RSLVQ	62.21 ± 1.5	56.48 ± 5.8	55.84 ± 4.9	55.48 ± 4.4	55.70 ± 3.9	56.31 ± 3.8	52.41 ± 10.2	49.43 ± 12.3	70.30 ± 0.3	60.86 ± 0.2	57.50(12)	11.4
ARF	84.80 ± 0.4	69.05 ± 4.6	87.49 ± 1.5	83.25 ± 1.8	78.15 ± 1.0	77.49 ± 0.9	65.56 ± 1.0	65.11 ± 0.9	97.25 ± 0.2	82.54 ± 0.1	79.06(5)	4.6
UOB	80.70 ± 0.4	69.27 ± 11.9	72.15 ± 10.6	72.92 ± 9.3	70.90 ± 9.3	70.45 ± 8.5	67.35 ± 10.9	64.97 ± 12.0	84.59 ± 0.0	76.81 ± 0.1	73.01(8)	8.0
OB	78.70 ± 0.9	57.66 ± 2.8	83.61 ± 1.7	80.58 ± 1.7	63.89 ± 0.5	70.02 ± 0.4	47.43 ± 0.6	46.84 ± 0.7	85.11 ± 0.1	76.51 ± 0.1	69.03(9)	9.6
RUSB	77.72 ± 0.7	57.57 ± 2.5	86.64 ± 1.5	83.53 ± 1.4	63.85 ± 0.4	69.24 ± 0.4	5.70 ± 0.9	5.99 ± 1.0	85.04 ± 0.1	75.39 ± 0.1	61.06(11)	9.3
AMF	83.65 ± 0.5	66.19 ± 3.4	90.29 ± 1.5	84.66 ± 2.0	67.78 ± 7.9	76.38 ± 0.7	50.48 ± 1.0	50.50 ± 1.1	96.60 ± 0.1	79.39 ± 0.5	74.59(7)	6.3
IWE	86.06 ± 0.4	63.43 ± 4.9	69.33 ± 3.3	67.53 ± 2.7	73.89 ± 1.5	83.98 ± 0.4	67.93 ± 0.5	67.33 ± 0.4	88.21 ± 1.5	80.23 ± 0.1	74.79(6)	5.6
F1score	SEAA	RTG	RBF	RBFr	AGRa	HYP	LEDA	LEDg	Sine	Waveform	AvgF1	AvgRank
CS	83.24 ± 0.4	68.00 ± 5.9	84.80 ± 1.7	80.73 ± 1.9	82.84 ± 0.9	78.28 ± 0.4	66.81 ± 0.6	65.86 ± 0.4	94.51 ± 0.0	83.67 ± 0.1	78.87(3)	4.2
CES	83.16 ± 0.6	67.93 ± 5.7	84.55 ± 1.7	80.66 ± 2.1	82.98 ± 0.8	78.13 ± 0.5	66.49 ± 0.5	65.78 ± 0.4	94.51 ± 0.1	83.69 ± 0.1	78.78(4)	4.9
CES-W	83.24 ± 0.5	68.04 ± 5.8	84.69 ± 1.6	80.74 ± 1.9	83.54 ± 0.9	78.09 ± 0.7	66.84 ± 0.3	65.90 ± 0.3	94.48 ± 0.0	83.65 ± 0.1	78.92(2)	4.2
TS-DM	84.17 ± 0.5	70.41 ± 5.6	86.14 ± 1.6	81.69 ± 1.8	85.77 ± 1.6	79.77 ± 0.6	67.85 ± 0.6	66.93 ± 0.6	95.97 ± 0.1	84.66 ± 0.1	80.33(1)	2.3
SRP	83.60 ± 0.4	60.81 ± 8.3	74.50 ± 2.3	72.67 ± 2.5	75.05 ± 1.5	71.99 ± 0.7	53.08 ± 3.9	52.54 ± 3.6	98.83 ± 0.1	16.65 ± 0.0	65.97(10)	7.1
RSLVQ	61.42 ± 1.5	55.82 ± 5.7	55.16 ± 4.8	54.78 ± 4.2	55.13 ± 3.9	55.83 ± 3.9	52.01 ± 10.0	49.10 ± 12.1	70.23 ± 0.3	60.43 ± 0.2	56.99(12)	11.2
ARF	83.46 ± 0.5	64.06 ± 7.2	87.02 ± 1.5	82.43 ± 2.3	78.15 ± 1.0	77.49 ± 0.9	65.24 ± 1.0	64.74 ± 0.9	97.25 ± 0.2	82.39 ± 0.1	78.22(5)	4.4
UOB	77.48 ± 0.7	62.46 ± 15.4	67.23 ± 14.4	68.90 ± 12.9	67.07 ± 12.1	66.88 ± 11.0	64.22 ± 12.1	62.15 ± 12.6	84.43 ± 0.0	76.01 ± 0.1	69.68(8)	8.9
OB	77.29 ± 0.9	55.75 ± 1.4	83.23 ± 1.7	80.12 ± 1.6	63.89 ± 0.5	70.00 ± 0.4	47.97 ± 0.5	47.37 ± 0.6	85.11 ± 0.1	76.47 ± 0.1	68.72(9)	9.5
RUSB	76.61 ± 0.7	56.09 ± 1.5	86.34 ± 1.5	83.13 ± 1.6	63.74 ± 0.4	69.23 ± 0.4	5.95 ± 0.9	6.25 ± 0.9	85.03 ± 0.1	75.38 ± 0.1	60.77(11)	9.0
AMF	82.04 ± 0.5	57.46 ± 9.8	89.98 ± 1.6	84.04 ± 2.3	66.35 ± 9.4	76.36 ± 0.7	49.69 ± 0.9	49.73 ± 1.1	96.60 ± 0.1	78.88 ± 0.6	73.11(7)	6.3
IWE	84.79 ± 0.4	54.68 ± 10.5	67.87 ± 3.9	65.72 ± 3.2	73.87 ± 1.5	83.98 ± 0.4	67.90 ± 0.5	67.28 ± 0.4	88.21 ± 1.5	79.10 ± 0.1	73.34(6)	5.8

shown in Table I. First, we generate datasets with sudden drift, such as the SEAA dataset [43] and the AGRa dataset [44]. Second, we generate datasets with incremental drift using the radial basis function (RBF) generator, like the RBF and RBFr datasets. The HYP dataset [43] contains incremental drift from the hyperplane generator. Next, we generate the RTG dataset [45] with no concept drift. In addition, the LEDa and LEDg datasets are two multiple-class datasets that contain sudden and gradual drift. In addition, we also choose two datasets from the River Library [46], namely, waveform (no drift) and sine (sudden drift). The chunk size of all synthetic data is set to $n = 100$ by default.

2) *Real-World Datasets*: As shown in Table I, we choose commonly used real-world data streams like electricity, weather, spam [47], EEG [48], airline, powersupply, PokerHand, and Usenet1 and Usenet2 [47] for the experiment. In addition, Covertype, INSECTSi (incremental drift) [49], and INSECTSa (abrupt drift) datasets [49] are also chosen for evaluation. Note that the chunk size of the Usenet1 and Usenet2 datasets is set as $n = 40$, since the number of data instances is relatively small. And the chunk size of the weather dataset is $n = 365$, since the data is collected daily over years. The chunk size of the other datasets is set to $n = 100$ by default.

The experiment was repeated 15 times on synthetic datasets and 5 times on real-world datasets with different random seeds. Python 3.7 was used to implement the proposed method. The computational environment was a Red Hat Enterprise Linux Workstation Release 7.9 (Maipo), with Intel Xeon Gold 6238R CPU @ 2.20 GHz. The state-of-the-art algorithms were implemented using the Scikit multflow meta module [50], with default parameter settings. The datasets and the Python source code are available online.¹

¹<https://github.com/kunkun111/TS-DM>

D. Experiment Results Discussion

1) *Experiment 1—Test the Chunk-Based Segmentation Strategy and the CES Strategy on Synthetic Datasets*: In this experiment, we tested the proposed chunk-based segmentation strategy and CES strategy on synthetic datasets. As shown in Table II, the chunk-based segmentation strategy performed better on the SEAA dataset, and its performance was better than that of the CES strategy when dealing with sudden drift. The average performance results of these two strategies showed insignificant difference. However, if different data distributions occurs in a single data chunk, the outcomes might vary. So, we tested these two strategies on real-world datasets.

2) *Experiment 2—Test the Chunk-Based Segmentation Strategy and the CES Strategy on Real-World Datasets*: In this experiment, we tested the chunk-based segmentation strategy and the CES strategy on the real-world datasets, the results are shown in Table III. The CES strategy performed better than the chunk-based segmentation strategy on some datasets, like weather and EEG datasets. It reflects that the CES strategy can help reduce the learning error when there are significant data distribution changes in a single data chunk. Blindly learning data chunks by using the chunk-based segmentation strategy is sometimes not enough for real-world application requirements.

3) *Experiment 3—Test the CES-W Process on Synthetic Datasets and Real-World Datasets*: In this experiment, we tested the CES-W method on synthetic datasets and real-world datasets, then compared and discussed the experimental results. As demonstrated in Tables II and III, the accuracy and F1 score of CES-W on some datasets showed improvement compared to CES, but not significantly. The performance difference between two methods depends on the frequency of the warning level data detected. CES-W is designed to identify and address the warning level data. Since warning level data may not occur at every time point, the performance difference

TABLE III
AVERAGE ACCURACY AND MACRO F1 SCORE WITH THE REAL-WORLD DATASETS (%), TS-DM GOT A HIGHER-AVERAGE PERFORMANCE

Accuracy	Electricity	Weather	Usenet1	Usenet2	Spam	Airline	EEG	Powersupply	Poker-Hand	Covertime	INSECTSi	INSECTSa	AvgAcc	AvgRank
CS	82.27±0.2	77.77±0.1	78.46±1.4	72.31±0.2	92.70±0.2	64.10±0.0	93.74±0.1	16.65±0.1	51.06±0.6	86.89±0.1	76.35±0.0	78.14±0.0	72.53(2)	3.8
CES	81.32±0.2	78.00±0.2	76.84±0.5	71.56±0.4	92.87±0.2	64.09±0.0	94.24±0.0	16.43±0.1	50.33±0.6	86.72±0.3	76.32±0.0	78.09±0.0	72.23(4)	4.7
CES-W	82.01±0.3	77.96±0.2	78.36±0.8	71.82±0.6	92.44±0.2	64.08±0.0	93.19±0.1	16.51±0.2	50.84±0.7	87.28±0.2	76.42±0.0	78.17±0.0	72.42(3)	4.3
TS-DM	82.30±0.1	78.46±0.1	78.50±0.7	72.50±1.1	93.53±0.1	65.38±0.0	94.65±0.1	15.84±0.2	51.36±0.8	93.71±0.0	77.96±0.0	79.68±0.1	73.65(1)	2.1
SRP	79.18±0.4	76.07±0.1	60.53±0.9	68.38±0.3	84.81±0.5	66.61±0.0	87.65±0.5	17.57±0.1	50.88±0.1	72.99±0.5	74.62±0.2	75.68±0.1	67.91(7)	6.3
RSLVQ	52.95±0.0	51.53±0.0	63.35±0.0	62.19±0.0	80.12±0.0	50.46±0.0	86.08±0.0	15.85±0.0	10.68±0.0	54.09±0.0	37.44±0.0	38.60±0.0	50.27(12)	10.6
ARF	79.85±0.2	78.22±0.2	65.63±0.7	70.69±0.7	91.75±0.1	66.82±0.0	89.07±0.3	16.74±0.1	53.60±0.2	92.28±0.0	78.75±0.0	79.86±0.0	71.93(5)	3.2
UOB	64.54±0.0	70.88±0.2	56.65±0.6	53.26±0.3	85.00±0.1	56.05±0.0	78.76±0.3	19.30±0.1	46.02±0.0	68.28±0.1	65.45±0.0	67.17±0.0	60.94(11)	9.0
OB	65.18±0.1	72.44±0.1	61.02±0.8	65.30±1.5	88.95±0.0	55.62±0.0	82.19±1.1	2.59±0.2	47.77±0.0	67.69±0.1	62.75±0.0	65.70±0.0	61.43(10)	9.3
RUSB	64.32±0.1	70.92±0.1	60.80±2.1	67.99±0.4	89.70±0.2	55.72±0.0	82.41±0.4	7.03±0.1	47.56±0.0	66.32±0.0	64.47±0.0	61.74±0.3	61.58(9)	9.4
AMF	73.30±0.1	78.16±0.1	60.09±0.9	69.56±0.6	92.96±0.2	58.88±0.1	76.19±0.6	11.47±0.2	54.58±0.2	89.89±0.1	76.70±0.0	75.40±0.1	68.09(6)	6.0
IWE	74.97±0.0	71.76±0.0	55.68±0.0	66.51±0.0	86.72±0.0	61.96±0.0	76.12±0.0	18.22±0.0	48.82±0.0	64.45±0.0	63.36±0.0	64.90±0.0	62.78(8)	8.8
F1 score	Electricity	Weather	Usenet1	Usenet2	Spam	Airline	EEG	Powersupply	Poker-Hand	Covertime	INSECTSi	INSECTSa	AvgF1	AvgRank
CS	81.80±0.2	73.30±0.1	78.33±1.5	65.02±0.3	90.30±0.3	63.43±0.0	93.70±0.1	16.41±0.1	11.37±0.1	78.56±0.0	63.38±0.0	67.13±0.0	65.22(2)	2.9
CES	80.81±0.2	73.54±0.0	63.07±0.0	58.98±0.0	66.43±0.0	64.40±0.5	90.53±0.0	16.17±0.1	11.19±0.1	78.54±0.1	63.32±0.0	67.07±0.0	64.39(12)	4.0
CES-W	81.52±0.3	73.48±0.3	78.12±0.9	64.68±0.6	89.93±0.3	63.40±0.0	93.14±0.1	16.28±0.2	11.30±0.1	79.23±0.4	63.48±0.1	67.10±0.0	65.13(3)	3.6
TS-DM	81.84±0.1	73.92±0.1	78.28±0.9	66.75±2.1	91.32±0.1	64.65±0.0	94.59±0.1	15.58±0.2	11.39±0.1	87.92±0.1	65.21±0.0	68.85±0.0	66.69(1)	1.5
SRP	78.42±0.4	67.98±0.1	59.91±0.7	47.72±0.9	76.19±1.1	64.54±0.0	87.50±0.5	13.81±0.1	7.41±0.0	56.84±0.9	58.69±0.1	63.21±0.2	56.85(7)	7.8
RSLVQ	51.44±0.0	50.80±0.0	63.07±0.0	58.98±0.0	66.43±0.0	50.27±0.0	85.95±0.0	15.06±0.0	4.65±0.0	43.95±0.0	32.48±0.0	33.66±0.0	46.39(12)	10.0
ARF	79.15±0.2	72.19±0.3	65.21±0.9	58.12±1.1	88.75±0.2	65.29±0.0	88.94±0.3	16.01±0.1	10.01±0.1	85.46±0.1	63.19±0.1	67.47±0.1	63.31(5)	4.6
UOB	59.56±0.0	49.65±0.7	55.40±1.0	53.23±0.3	76.25±0.3	55.19±0.0	78.56±0.3	17.62±0.1	8.89±0.0	53.44±0.1	48.93±0.0	53.40±0.1	50.84(11)	9.3
OB	63.82±0.1	65.59±0.2	60.86±0.9	59.73±1.2	84.47±0.0	55.15±0.0	81.92±1.1	3.01±0.2	11.28±0.0	53.70±0.0	51.50±0.0	55.80±0.1	53.90(8)	8.3
RUSB	63.28±0.1	65.44±0.0	60.42±2.1	61.59±0.9	85.49±0.3	54.77±0.0	82.18±0.4	6.86±0.1	11.35±0.0	52.91±0.0	50.64±0.0	49.98±0.4	53.74(9)	8.5
AMF	72.35±0.1	71.28±0.4	60.07±0.9	55.07±1.5	90.50±0.3	48.54±0.3	75.98±0.6	11.36±0.2	9.27±0.1	83.14±0.1	59.00±0.0	57.25±0.1	57.81(6)	7.7
IWE	74.00±0.0	66.73±0.0	54.97±0.0	39.94±0.0	82.31±0.0	60.58±0.0	75.80±0.0	14.19±0.0	8.12±0.0	50.32±0.0	55.79±0.0	57.75±0.0	53.37(10)	9.2

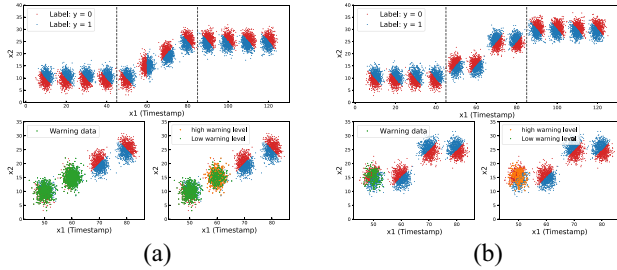


Fig. 4. Simple visualization of the warning level data segmentation process on generated data with incremental and sudden drift. The warning level data can be identified and segmented under incremental drift, but not very obvious in handling sudden drift. (a) Data S1 with incremental drift. (b) Data S2 with sudden drift.

between these two methods is insignificant. Moreover, to better understand the segmentation process of CES-W, we also visualized the results on the newly generated datasets S1 and S2, and the segmented high-warning level data and low-warning level data under both incremental drift and sudden drift have been marked, as shown in Fig. 4. Consequently, this process can segment the warning level data to support model learning.

4) *Experiment 4—Test the TS-DM on Synthetic Datasets and Real-World Datasets:* In this experiment, we tested the TS-DM on both synthetic and real-world datasets, the results are shown in Tables II–IV, Figs. 5 and 6, and Appendix A. The results show that AMF and IWE perform well on some datasets since they tend to incrementally utilize historical data for model adaptation. This benefits data stream with low-drift severity like RBF and HYP. However, this approach proves less effective on datasets with high-drift severity, such as AGRA, where outdated concepts cannot help model adaptation. Instead, the proposed TS-DM aims to ensure the balance between model accuracy and learning robustness.

TABLE IV
MCC SCORE COMPARISON RESULTS (%)

Method	TS-DM	SRP	RSLVQ	ARF	UOB	OB	RUSB	AMF	IWE
SEAA	68.56±1.0	67.55±0.7	24.08±2.8	67.35±0.9	59.06±1.2	54.61±1.8	53.31±1.5	64.82±1.1	70.10±0.9
RTG	41.34±11.3	26.99±10.8	1.51±1.3	33.00±10.3	10.73±4.7	11.54±2.9	12.50±3.1	24.64±12.6	17.01±13.1
RBF	72.32±3.2	50.61±4.1	7.77±2.2	74.36±3.1	57.93±4.5	66.51±3.4	72.74±3.1	80.07±3.1	36.50±6.9
RBF	63.49±3.6	47.04±3.7	7.38±2.5	65.58±4.0	52.73±4.2	60.30±3.3	66.33±3.2	68.45±4.3	32.34±6.0
AGRA	71.58±3.2	50.15±3.2	13.27±1.9	56.33±2.1	30.77±1.0	10.27±0.8	10.27±0.8	37.50±15.1	47.88±3.1
HYP	59.54±1.3	44.01±1.5	18.68±0.9	55.00±1.8	41.72±1.0	40.03±0.9	38.58±0.9	52.77±1.5	67.96±0.9
LEDa	64.32±0.7	49.50±4.1	21.11±0.6	61.76±1.1	43.44±0.8	41.65±0.7	4.84±1.0	52.03±2.7	71.15±1.3
LEDg	63.31±0.7	49.02±3.8	20.66±0.6	61.26±1.0	42.90±0.7	40.99±0.8	4.51±1.1	51.77±2.5	70.50±1.3
Sine	91.96±0.3	97.67±0.2	40.80±0.6	94.52±0.4	70.62±0.1	70.22±0.2	70.11±0.2	93.20±0.3	76.51±3.1
Waveform	77.07±0.1	0.00±0.0	41.45±0.3	74.01±0.1	66.99±0.2	64.78±0.1	63.10±0.2	72.79±0.6	76.02±0.2
AvgScore	67.34	48.25	19.67	64.31	47.68	47.84	39.52	59.80	56.59
AvgRank	2.3	4.8	8.7	2.7	6.1	6.6	6.7	3.5	3.6
Electricity	63.70±0.2	57.08±0.9	2.95±0.0	58.49±0.4	25.21±0.1	27.90±0.3	26.58±0.2	44.89±0.2	48.28±0.0
Weather	48.25±0.3	39.79±0.3	8.52±0.0	46.30±0.6	20.75±1.1	32.21±0.4	31.00±0.2	45.72±0.5	33.51±0.0
Usenet1	57.10±1.4	20.46±1.7	26.25±0.0	30.77±1.6	16.97±0.7	22.02±2.1	21.05±4.3	20.39±2.0	10.45±0.0
Usenet2	34.66±1.6	16.81±2.2	18.35±0.0	26.27±2.4	20.58±1.4	19.95±2.5	24.14±1.5	22.01±2.5	2.62±0.0
Spam	82.65±0.2	55.68±1.9	38.84±0.0	77.55±0.4	56.25±0.4	66.81±1.5	71.37±0.6	81.02±0.6	64.62±0.0
Airline	29.46±0.0	31.63±0.1	0.78±0.0	32.01±0.2	17.79±0.0	10.31±0.1	9.69±0.0	14.99±0.5	21.94±0.0
EEG	89.20±0.2	75.03±1.0	71.90±0.1	77.92±0.7	57.13±0.7	63.96±2.2	64.43±0.8	51.97±1.2	51.66±0.0
Powersupply	12.19±0.2	14.23±0.1	12.22±0.0	13.14±0.1	15.94±0.1	1.77±0.2	3.01±0.1	11.50±0.4	29.19±0.0
PokerHand	10.09±1.4	11.45±4.6	0.29±0.0	11.84±0.6	5.28±0.0	5.73±0.0	5.43±0.0	12.68±0.4	0.10±0.0
Covertime	89.88±0.1	55.37±0.9	31.48±0.0	87.53±0.1	47.34±0.2	47.23±0.1	45.56±0.1	83.96±0.2	41.27±0.0
INSECTSi	71.30±0.0	66.75±0.2	22.11±0.0	72.25±0.0	54.86±0.0	51.68±0.0	53.73±0.0	73.67±0.1	54.39±0.0
INSECTSa	73.65±0.1	68.31±0.2	23.40±0.0	73.81±0.0	57.35±0.0	55.62±0.1	50.67±0.2	73.00±0.1	58.77±0.0
AvgScore	55.17	42.71	21.42	50.65	32.95	33.47	33.88	44.65	34.29
AvgRank	2.0	4.3	7.5	2.0	6.0	6.2	6.2	4.1	6.2

By selectively utilizing and discarding the historical data, our method demonstrates enhanced robustness, achieving the highest-average score and rank of accuracy, F1, and MCC. In addition, the TS-DM got higher-prequential accuracy on some datasets, which also shows its robustness. Finally, the critical difference (CD) diagrams and different statistical test results also demonstrate the efficiency of the proposed method, as shown in Table VI and Fig. 7. The proposed TS-DM can be applied to real-world scenarios to support decision-making.

TABLE V
RUNTIME COMPARISON RESULTS (UNIT: SECONDS)

Method	TS-DM	SRP	RSLVQ	ARF	Online UOB	OnlineBoost	RUSBoost	AMF	IWE
SEAA	1.53E+01	9.31E+01	2.86E+00	6.49E+01	4.23E+02	4.51E+02	5.15E+02	1.99E+01	5.43E+00
RTG	2.18E+01	5.41E+01	2.98E+00	8.27E+01	4.31E+02	4.69E+02	5.66E+02	2.67E+01	9.74E+00
RBF	2.70E+01	5.89E+01	3.04E+00	7.76E+01	4.29E+02	4.44E+02	5.09E+02	3.35E+01	1.05E+01
RBFr	2.76E+01	5.88E+01	3.05E+00	8.03E+01	4.28E+02	4.39E+02	5.16E+02	3.41E+01	1.05E+01
AGRa	1.79E+01	5.51E+01	3.04E+00	7.92E+01	4.30E+02	4.53E+02	5.60E+02	3.04E+01	9.62E+00
HYP	2.84E+01	5.70E+01	3.04E+00	8.24E+01	4.32E+02	4.91E+02	5.89E+02	2.90E+01	1.07E+01
LEDa	2.47E+02	1.69E+02	4.47E+00	2.82E+02	4.34E+02	5.51E+02	4.31E+02	4.75E+01	1.12E+02
LEDg	2.42E+02	1.75E+02	5.54E+00	3.00E+02	4.36E+02	5.42E+02	4.41E+02	4.68E+01	1.12E+02
Sine	1.87E+02	7.10E+02	2.86E+01	5.56E+02	3.62E+03	3.76E+03	4.31E+03	2.19E+02	5.19E+02
Waveform	3.04E+03	5.85E+02	4.90E+01	1.66E+03	4.64E+03	6.33E+03	8.08E+03	1.16E+03	6.42E+01
Electricity	8.19E+01	2.71E+02	1.16E+01	4.05E+02	1.92E+03	2.08E+03	2.55E+03	1.11E+02	3.93E+01
Weather	1.37E+01	1.13E+02	5.18E+00	1.88E+02	2.38E+03	2.42E+03	2.57E+03	6.18E+01	1.55E+01
Usenet1	4.48E+00	1.01E+01	5.89E-01	2.21E+01	9.78E+01	1.39E+02	2.20E+02	3.14E+01	6.70E+00
Usenet2	4.10E+00	9.98E+00	6.78E-01	2.14E+01	8.20E+01	1.34E+02	2.11E+02	3.08E+01	5.54E+00
Spam	5.24E+01	7.00E+01	5.91E+00	2.55E+02	4.15E+03	3.26E+03	3.19E+03	9.37E+02	2.47E+02
Airline	9.66E+02	3.09E+03	1.66E+02	5.50E+03	2.29E+04	2.54E+04	2.87E+04	2.36E+03	3.85E+02
EEG	2.89E+01	7.63E+01	3.53E+00	9.46E+01	8.72E+02	1.15E+03	1.01E+03	5.86E+01	1.40E+01
Powersupply	1.49E+03	1.40E+03	8.60E+01	1.05E+03	3.21E+03	3.59E+03	2.92E+03	1.70E+02	1.34E+02
PokerHand	1.50E+04	8.80E+03	1.34E+03	1.76E+04	4.58E+04	5.14E+04	5.90E+04	5.88E+03	1.56E+03
Covertyp	1.02E+04	3.22E+03	4.83E+02	6.19E+03	3.07E+04	3.24E+04	3.87E+04	8.35E+03	3.05E+03
INSECTSi	2.42E+04	3.72E+03	7.27E+02	9.09E+03	2.73E+04	2.49E+04	2.11E+04	6.73E+03	4.10E+03
INSECTSa	1.85E+04	2.88E+03	5.59E+02	7.24E+03	1.48E+04	1.87E+04	3.30E+04	5.00E+03	3.07E+03

TABLE VI
RESULTS OF FRIEDMAN TEST, WILCOXON SIGNED-RANK TEST, AND STUDENT T-TEST (UNDER 0.05 SIGNIFICANCE LEVEL)

	Method	SRP	RSLVQ	ARF	OUB	OB	RUSB	AMF	IWE
Friedman Test	$p(\text{Acc})$, synthetic	0.01141	0.00157	0.2059	0.00157	0.00157	0.05778	0.2059	0.52709
	$p(\text{Fi})$, synthetic	0.01141	0.00157	0.2059	0.00157	0.00157	0.05778	0.2059	0.52709
	$p(\text{Acc})$, real world	0.02092	0.00389	0.5637	0.00389	0.00053	0.00053	0.00389	0.00389
	$p(\text{Fi})$, real world	0.00053	0.00053	0.02092	0.00389	0.00053	0.00053	0.00053	0.00053
Wilcoxon Signed-Rank Test	$p(\text{Acc})$, synthetic	0.00932	0.00512	0.05876	0.00512	0.00512	0.01242	0.05876	0.07508
	$p(\text{Fi})$, synthetic	0.00932	0.00512	0.05876	0.00512	0.00512	0.01242	0.05876	0.07508
	$p(\text{Acc})$, real world	0.00758	0.00288	0.20766	0.00288	0.00222	0.00222	0.00758	0.00288
	$p(\text{Fi})$, real world	0.00222	0.00222	0.0048	0.00288	0.00222	0.00222	0.00222	0.00222
Student T-Test	$p(\text{Acc})$, synthetic	0.042162	0.00001	0.35311	0.019702	0.022024	0.03349	0.149812	0.080975
	$p(\text{Fi})$, synthetic	0.037633	0.00001	0.322525	0.005293	0.02194	0.033516	0.122212	0.066486
	$p(\text{Acc})$, real world	0.249447	0.008164	0.423015	0.063785	0.091224	0.087348	0.267656	0.095426
	$p(\text{Fi})$, real world	0.177595	0.027293	0.37797	0.057778	0.116765	0.111732	0.206895	0.102569

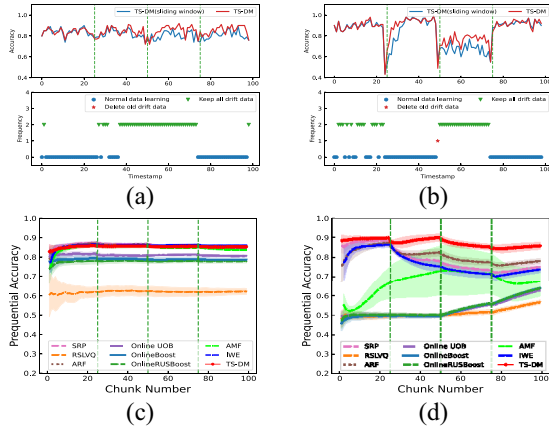


Fig. 5. Illustration of the learning accuracy of TS-DM and benchmark methods on SEAA and AGRa datasets. (a) SEAA (chunk accuracy). (b) AGRa (chunk accuracy). (c) SEAA (prequential accuracy). (d) AGRa (prequential accuracy).

5) *Experiment 5—Compare With the TS-DM Based on Sliding Window (SW)*: Since the proposed TS-DM selectively retains or deletes the earlier historical data, we compare its performance with the TS-DM embedded with SW strategy, which is a traditional time series segmentation strategy that discards the historical data after the new data arrives. We also compare the accuracy of these two methods, as shown in Fig. 8. The results show that our proposed TS-DM performs

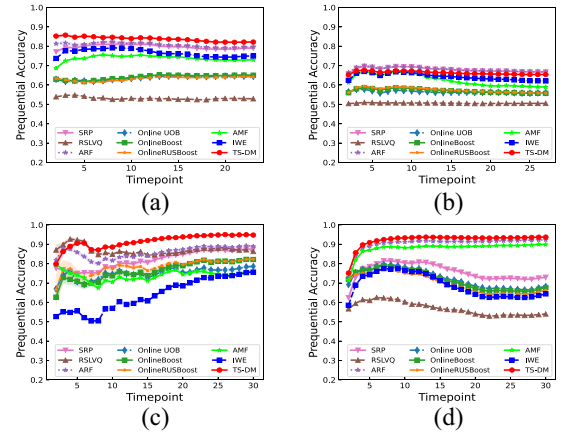


Fig. 6. Illustration of the prequential accuracy of the TS-DM and benchmark methods on real-world datasets. (a) Electricity. (b) Airline. (c) EEG. (d) Covertyp.

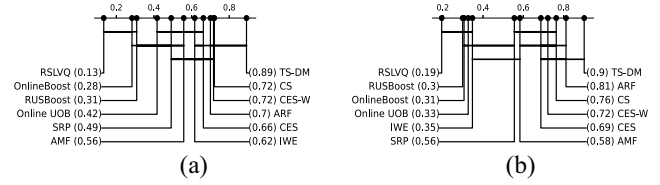


Fig. 7. Results of the CD diagrams show the percentile rank of the TS-DM and benchmark methods. (a) Results on synthetic datasets. (b) Results on real-world datasets.

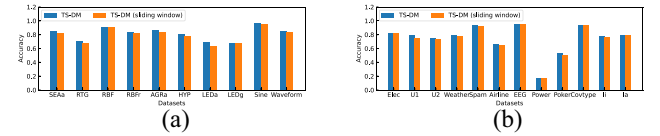


Fig. 8. Accuracy of the TS-DM and the TS-DM with the SW method. (U1: Usenet1, U2: Usenet2, Ii: INSECTSi, and Ia: INSECTSa). (a) Synthetic data comparison. (b) Real-world data comparison.

relatively well on most datasets. However, after embedding the SW method with a fixed window size, the learning performance of TS-DM has not improved significantly, especially in handling different drift situations. The main reason is that this method does not appropriately manage the retention or discarding of historical data, thus resulting in limited enhancement of model performance. This comparison verifies the flexibility and efficiency of TS-DM.

6) *Experiment 6—Time Cost Analysis*: The runtime complexity of TS-DM includes two parts: 1) time segmentation and 2) the retraining process. As shown in Table V, the AMF and IWE methods, implemented in the River Library with efficient parallelized computation and optimized implementations, result in minimal time cost. While our method lacks similar code optimization, it still outperforms other benchmark methods.

7) *Experiment 7—Parameter Sensitivity Analysis*: An important parameter is the previously mentioned β , $\beta \in (0, 1)$, which is a manually set parameter to reflect the accuracy decrease under high-severity concept drift. We set this value as $\{0.6, 0.7, 0.8, 0.9, 1\}$ to test the proposed method and output

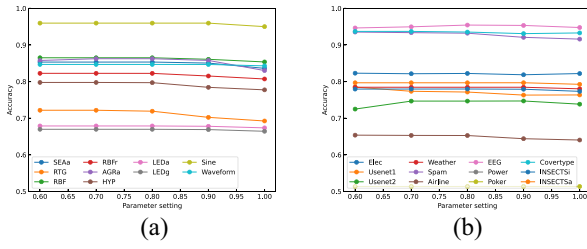


Fig. 9. Plot of the parameter sensitivity analysis. (a) Synthetic data comparison. (b) Real-world data comparison.

the accuracy score. With this change, the model accuracy changed on the SEAA and AGRA datasets, which reflects the high–low-drift severity tradeoff. In our experiment, we choose the parameters as $\beta = 0.6$ which leads to a better performance, and the results are shown in Fig. 9. Furthermore, we also conduct a detailed analysis of three additional parameters: 1) chunk size n ; 2) number of base learners m ; and 3) type of learning model $F(x)$, as shown in Appendix B.

8) *Discussion*: The experiment results show the effectiveness of the proposed method.

- 1) Appropriately segmenting and combining streaming data can help reduce learning error, which is the requirement of Problem 1.
- 2) Segmenting normal and drift data helps reduce impact when old and new concepts coexist in a single chunk that Problem 2 concerns. Based on this, the warning level data segmentation process further maintains the performance.
- 3) Finally, selectively keep or discard data with different drift severity maintains the learning robustness on different drift situations that Problem 3 requires.

VI. CONCLUSION

In this article, a TS-DM is proposed to support data stream learning under concept drift. First, a chunk-based segmentation strategy and a CES strategy have been introduced to help segment and group the streaming data. Then, to maintain learning performance, a warning level data segment process has been designed to divide the warning level data into low-warning-level data and high-warning-level data. Next, to ensure the model can deal well with different drift severity, a further improved learning process has been developed that can selectively keep or give up the data according to the drift severity. The experiment results show the efficiency of the proposed method.

In our future research, we will concentrate on enhancing the flexibility and generalization of the proposed learning method to further elevate its performance. In addition, we also aim to expand the method to address different drift scenarios, enabling model learning to be more dynamic and automated.

REFERENCES

[1] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.

[2] A. Liu, J. Lu, and G. Zhang, “Concept drift detection via equal intensity k-means space partitioning,” *IEEE Trans. Cybern.*, vol. 51, no. 6, pp. 3198–3211, Jun. 2021.

[3] S. U. Din, J. Kumar, J. Shao, C. B. Mawuli, and W. D. Ndiaye, “Learning high-dimensional evolving data streams with limited labels,” *IEEE Trans. Cybern.*, vol. 52, no. 11, pp. 11373–11384, Nov. 2022.

[4] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Comput. Surv.*, vol. 46, no. 4, pp. 1–37, 2014.

[5] Y. Sun, K. Tang, Z. Zhu, and X. Yao, “Concept drift adaptation by exploiting historical knowledge,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 10, pp. 4822–4832, Oct. 2018.

[6] F. Dong, J. Lu, Y. Song, F. Liu, and G. Zhang, “A drift region-based data sample filtering method,” *IEEE Trans. Cybern.*, vol. 52, no. 9, pp. 9377–9390, Sep. 2022.

[7] E. Fuchs, T. Gruber, J. Nitschke, and B. Sick, “Online segmentation of time series based on polynomial least-squares approximations,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 12, pp. 2232–45, Dec. 2010.

[8] X. Huang, M. Matijas, and J. A. K. Suykens, “Hinging hyperplanes for time-series segmentation,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 24, no. 8, pp. 1279–1291, Aug. 2013.

[9] U. Appel and A. V. Brandt, “Adaptive sequential segmentation of piecewise stationary time series,” *Inf. Sci.*, vol. 29, no. 1, pp. 27–56, 1983.

[10] D. Guijo-Rubio, A. M. Durán-Rosal, P. A. Gutiérrez, A. Troncoso, and C. Hervás-Martínez, “Time-series clustering based on the characterization of segment typologies,” *IEEE Trans. Cybern.*, vol. 51, no. 11, pp. 5409–5422, Nov. 2021.

[11] D. Gong, G. Medioni, and X. Zhao, “Structured time series analysis for human action segmentation and recognition,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 36, no. 7, pp. 1414–1427, Jul. 2014.

[12] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, “Learning with drift detection,” in *Proc. 17th Brazilian Symp. Artif. Intell.*, 2004, pp. 286–295.

[13] K. Wang, J. Lu, A. Liu, G. Zhang, and L. Xiong, “Evolving gradient boost: A pruning scheme based on loss improvement ratio for learning under concept drift,” *IEEE Trans. Cybern.*, vol. 53, no. 4, pp. 2110–2123, Apr. 2023.

[14] G. Ditzler, M. Roveri, C. Alippi, and R. Polikar, “Learning in nonstationary environments: A survey,” *IEEE Comput. Intell. Mag.*, vol. 10, no. 4, pp. 12–25, Nov. 2015.

[15] Ö. Gözdağcı and F. Can, “Concept learning using one-class classifiers for implicit drift detection in evolving data streams,” *Artif. Intell. Rev.*, vol. 54, pp. 3725–3747, Jun. 2021.

[16] S. Khezri, J. Tanha, A. Ahmadi, and A. Sharifi, “STDS: Self-training data streams for mining limited labeled data in non-stationary environment,” *Appl. Intell.*, vol. 50, pp. 1448–1467, Jan. 2020.

[17] D. M. V. Sato, S. C. De Freitas, J. P. Barddal, and E. E. Scalabrín, “A survey on concept drift in process mining,” *ACM Comput. Surv.*, vol. 54, no. 9, pp. 1–38, 2021.

[18] B. Krawczyk, L. L. Minku, J. Gama, J. Stefanowski, and M. Wozniak, “Ensemble learning for data stream analysis: A survey,” *Inf. Fusion*, vol. 37, pp. 132–156, Sep. 2017.

[19] S. Pan, J. Wu, X. Zhu, and C. Zhang, “Graph ensemble boosting for imbalanced noisy graph stream classification,” *IEEE Trans. Cybern.*, vol. 45, no. 5, pp. 954–968, May 2015.

[20] M. Pratama, E. Dimla, T. Tjahjowidodo, W. Pedrycz, and E. Lughofer, “Online tool condition monitoring based on parsimonious ensemble+,” *IEEE Trans. Cybern.*, vol. 50, no. 2, pp. 664–677, Feb. 2020.

[21] A. Liu, J. Lu, and G. Zhang, “Diverse instance-weighting ensemble based on region drift disagreement for concept drift adaptation,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 293–307, Jan. 2021.

[22] H. M. Gomes et al., “Adaptive random forests for evolving data stream classification,” *Mach. Learn.*, vol. 106, nos. 9–10, pp. 1469–1495, 2017.

[23] N. C. Oza, “Online bagging and boosting,” in *Proc. IEEE Int. Conf. Syst., Man Cybern.*, 2005, pp. 2340–2345.

[24] B. Wang and J. Pineau, “Online bagging and boosting for imbalanced data streams,” *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 12, pp. 3353–3366, Dec. 2016.

[25] J. Sui, Z. Liu, L. Liu, A. Jung, and X. Li, “Dynamic sparse subspace clustering for evolving high-dimensional data streams,” *IEEE Trans. Cybern.*, vol. 52, no. 6, pp. 4173–4186, Jun. 2022.

[26] H. Yu, J. Lu, and G. Zhang, “Continuous support vector regression for nonstationary streaming data,” *IEEE Trans. Cybern.*, vol. 52, no. 5, pp. 3592–3605, May 2022.

- [27] J. Lu, H. Zuo, and G. Zhang, "Fuzzy multiple-source transfer learning," *IEEE Trans. Fuzzy Syst.*, vol. 28, no. 12, pp. 3418–3431, Dec. 2020.
- [28] J. Mourtada, S. Gaïffas, and E. Scornet, "AMF: Aggregated Mondrian forests for online learning," *J. Roy. Stat. Soc. Ser. B, Stat. Methodol.*, vol. 83, no. 3, pp. 505–533, 2021.
- [29] B. Jiao, Y. Guo, C. Yang, J. Pu, Z. Zheng, and D. Gong, "Incremental weighted ensemble for data streams with concept drift," *IEEE Trans. Artif. Intell.*, vol. 5, no. 1, pp. 92–103, Jan. 2024.
- [30] H. Yu, W. Liu, J. Lu, Y. Wen, X. Luo, and G. Zhang, "Detecting group concept drift from multiple data streams," *Pattern Recognit.*, vol. 134, Feb. 2023, Art. no. 109113.
- [31] M. Zhou, J. Lu, Y. Song, and G. Zhang, "Multi-stream concept drift self-adaptation using graph neural network," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12828–12841, Dec. 2023.
- [32] F. L. Chung, T. C. Fu, V. Ng, and R. Luk, "An evolutionary approach to pattern-based time series segmentation," *IEEE Trans. Evol. Comput.*, vol. 8, no. 5, pp. 471–489, Oct. 2004.
- [33] F. Liu et al., "Anomaly detection in quasi-periodic time series based on automatic data segmentation and attentional LSTM-CNN," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 6, pp. 2626–2640, Jun. 2022.
- [34] C. Xiao, S. Chen, F. Zhou, and J. Wu, "Self-supervised few-shot time-series segmentation for activity recognition," *IEEE Trans. Mobile Comput.*, vol. 22, no. 11, pp. 6770–6783, Nov. 2023.
- [35] Y. Pang, M. Shi, L. Zhang, W. Sun, and X. Song, "A multivariate time series segmentation algorithm for analyzing the operating statuses of tunnel boring machines," *Knowl.-Based Syst.*, vol. 242, Apr. 2022, Art. no. 108362.
- [36] K. Wang, J. Lu, A. Liu, Y. Song, L. Xiong, and G. Zhang, "Elastic gradient boosting decision tree with adaptive iterations for concept drift adaptation," *Neurocomputing*, vol. 491, pp. 288–304, Jun. 2022.
- [37] H. M. Gomes, J. Read, and A. Bifet, "Streaming random patches for evolving data stream classification," in *Proc. IEEE Int. Conf. Data Min.*, 2019, pp. 240–249.
- [38] M. Heusinger, C. Raab, and F.-M. Schleif, "Passive concept drift handling via momentum based robust soft learning vector quantization," in *Proc. 13th Int. Workshop WSOM*, 2020, pp. 200–209.
- [39] K. Stapor, P. Ksieniewicz, S. Garcia, and M. Woźniak, "How to design the fair experimental classifier evaluation," *Appl. Soft Comput.*, vol. 104, Jun. 2021, Art. no. 107219.
- [40] D. J. Hand, P. Christen, and N. Kirielle, "F*: An interpretable transformation of the F-measure," *Mach. Learn.*, vol. 110, no. 3, pp. 451–456, 2021.
- [41] D. Brzezinski and J. Stefanowski, "Prequential AUC: Properties of the area under the ROC curve for data streams with concept drift," *Knowl. Inf. Syst.*, vol. 52, pp. 531–562, Jan. 2017.
- [42] A. Bifet, G. Holmes, R. Kirkby, and B. Pfahringer, "MOA: Massive online analysis," *J. Mach. Learn. Res.*, vol. 11, pp. 1601–1604, Apr. 2010.
- [43] W. N. Street and Y. Kim, "A streaming ensemble algorithm (SEA) for large-scale classification," in *Proc. 7th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2001, pp. 377–382.
- [44] R. Agrawal, T. Imielinski, and A. Swami, "Database mining: A performance perspective," *IEEE Trans. Knowl. Data Eng.*, vol. 5, no. 6, pp. 914–925, Dec. 1993.
- [45] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proc. 6th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2000, pp. 71–80.
- [46] J. Montiel et al., "River: Machine learning for streaming data in python," *J. Mach. Learn. Res.*, vol. 22, no. 1, pp. 4945–4952, 2021.
- [47] I. Katakis, G. Tsoumakas, and I. Vlahavas, "Tracking recurring contexts using ensemble classifiers: An application to email filtering," *Knowl. Inf. Syst.*, vol. 22, no. 3, pp. 371–391, 2010.
- [48] D. Dua and C. Graff, "UCI machine learning repository." 2017. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [49] V. M. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. Batista, "Challenges in benchmarking stream learning algorithms with real-world data," *Data Min. Knowl. Discov.*, vol. 34, pp. 1805–1858, Jul. 2020.
- [50] J. Montiel, J. Read, A. Bifet, and T. Abdesslem, "Scikit-multiflow: A multi-output streaming framework," *J. Mach. Learn. Res.*, vol. 19, no. 72, pp. 1–5, 2018.



Kun Wang received the Ph.D. degree in computer science from the University of Technology Sydney, Sydney, NSW, Australia, in 2024, and the Ph.D. degree in management science and engineering from Shanghai University, Shanghai, China, in 2023.

She is a Postdoctoral Research Fellow with Australian Artificial Intelligence Institute, University of Technology Sydney. She has published ten papers in related areas. Her research interests include concept drift adaptation, data stream mining, and information management.



Jie Lu (Fellow, IEEE) received the Ph.D. degree from Curtin University, Bentley, WA, Australia, in 2000.

She is an Australian Laureate Fellow, an IFSA Fellow, an ACS Fellow, a Distinguished Professor, and the Director of Australian Artificial Intelligence Institute, University of Technology Sydney, Sydney, NSW, Australia. She has published over 500 papers in IEEE TRANSACTIONS and other leading journals and conferences. Her main research expertise is in transfer learning, concept drift, fuzzy systems, decision support systems, and recommender systems.

Prof. Lu is the recipient of two IEEE Transactions on Fuzzy Systems Outstanding Paper Awards in 2019 and 2022, the NeurIPS2022 Outstanding Paper Award, the Australia's Most Innovative Engineer Award in 2019, the Australasian Artificial Intelligence Distinguished Research Contribution Award in 2022, the Australian NSW Premier's Prize on Excellence in Engineering or Information and Communication Technology in 2023, and the Officer of the Order of Australia in 2023.



Anjin Liu received the Bachelor of Information Technology degree (Hons.) from the University of Sydney, Sydney, NSW, Australia, in 2012.

He is a Postdoctoral Research Associate with Australian Artificial Intelligence Institute, University of Technology Sydney. He has published research papers on IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON FUZZY SYSTEMS, IEEE TRANSACTIONS ON CYBERNETICS, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, as well as IJCAI and IJCNN. His research interests include concept drift detection, adaptive data stream learning, multistream learning, machine learning, and big data analytics.



Guangquan Zhang received the Ph.D. degree in applied mathematics from the Curtin University of Technology, Bentley, WA, Australia, in 2001.

He is an Associate Professor and the Director of the Decision Systems and e-Service Intelligent Research Laboratory, University of Technology Sydney, Sydney, NSW, Australia. He has authored five monographs, five textbooks, and 460 papers, including 220 refereed international journal papers. His research interests include fuzzy machine learning, fuzzy optimization, and machine learning.

Prof. Zhang was awarded an ARC QEII Fellowship in 2005. He has won seven Australian Research Council Discovery Projects Grants and many other research grants. He has served as a member for the editorial boards of several international journals, as a Guest Editor of eight special issues for IEEE transactions and other international journals, and co-chaired several international conferences and workshops in the areas of fuzzy decision making and knowledge engineering.